Hovhannes Muradyan

Coding Assignment 1: Designing a Simple Single-Server Key-Value Store

When building the key-value store, I chose Python alongside the Flask micro-framework primarily because of my familiarity with the framework. I opted for a native Python dictionary to leverage its underlying hash map structure as it gives O(1) for all 3 operations and is more than enough for this scale project.

In terms of API architecture, I decided to treat the target key as a dynamic URL parameter (such as /<key>) while placing the actual payload data inside the JSON request body. Taking this RESTful approach keeps standard GET and DELETE requests clean and lightweight, avoiding the anti-pattern of forcing request bodies onto read operations. Positioning the key directly in the URL route also enables network components to inspect and route incoming traffic quickly without parsing the body payload.

To handle concurrency and ensure thread safety across Flask's multithreaded workers, I wrapped dictionary updates in a threading. Lock mutual exclusion lock to prevent race conditions during simultaneous writes. I also save everything to a local file after each “change” operation as a backup option for cases where restart is needed.

I assumed the overall size of the dataset would comfortably fit within the machine's physical memory boundaries. Second, I assumed that stored values consist of basic primitives or strings that easily serialize to JSON without requiring custom encoders. For bigger systems, better hash data structure and architecture with sharding depending on the scale.